

Assignment: E-Commerce Order Management System

Course Information

Subject: Service-Oriented Architecture

Assignment Type: Programming Project

Duration: 4-6 weeks

Team Size: 3-4 students (or individual)

Total Points: 100 points

Phases: 3

Learning Objectives

By completing this assignment, you will:

1. Design and implement a microservices-based application architecture
2. Develop RESTful APIs using Python Flask framework
3. Integrate Java JSP frontend with multiple backend microservices
4. Implement inter-service communication patterns¹
5. Design database schemas for distributed services
6. Apply service orchestration and composition principles

Project Overview

You will develop an **E-Commerce Order Management System** that demonstrates microservices architecture principles.

The system consists of a Java JSP web application (frontend) that communicates with **five Python Flask microservices** (backend).

Core Architecture Requirements:

- Java JSP application serves as the API gateway and user interface
- Five independent Python Flask services handle specific business logic
- Each service runs on a separate port
- Services communicate via HTTP REST APIs
- At least two services must interact with a MySQL database

System Requirements

Technology Stack

- **Frontend:** Java JSP with Jakarta EE (Servlets)
- **Backend Services:** Python 3.8+ with Flask framework
- **Database:** MySQL 8.0
- **Web Server:** Apache Tomcat 10.x
- **API Communication:** REST with JSON payloads^{[6][5]}

Port Configuration

Assign the following ports to avoid conflicts:

- Java JSP Application: 8080 (Tomcat default)
- Order Service: 5001
- Inventory Service: 5002
- Pricing Service: 5003
- Customer Service: 5004
- Notification Service: 5005

Microservice Specifications

Service 1: Order Service (30% of service grade)

Type: Receives all parameters from Java application^{[3][4]}

Responsibilities:

- Accept order creation requests from JSP application
- Validate all input parameters (customer ID, product IDs, quantities)
- Generate unique order ID and timestamp
- Return order confirmation with status

Required Endpoints:

- POST /api/orders/create - Create new order
- GET /api/orders/{order_id} - Retrieve order details

Input Parameters from JSP:

```
{
  "customer_id": integer,
  "products": [
    {"product_id": integer, "quantity": integer}
  ],
  "total_amount": decimal
}
```

 **Hint:** Use Flask's `request.get_json()` to parse incoming JSON data. Implement input validation using try-except blocks to handle missing or invalid parameters.^{[1][4]}

Service 2: Inventory Service (20% of service grade)

Type: Database-driven service

Responsibilities:

- Query MySQL database for product availability
- Update stock quantities after order placement
- Check inventory levels before order confirmation
- Maintain product catalog

Required Endpoints:

- GET /api/inventory/check/{product_id} - Check stock availability
- PUT /api/inventory/update - Update inventory after order

Database Schema:

```
CREATE TABLE inventory (
  product_id INT PRIMARY KEY AUTO_INCREMENT,
  product_name VARCHAR(100) NOT NULL,
  quantity_available INT NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL,
  last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

 **Hint:** Use the `mysql-connector-python` library to connect Flask to MySQL. Always use parameterized queries to prevent SQL injection. Example:

```
import mysql.connector
from flask import Flask, jsonify

conn = mysql.connector.connect(
  host="localhost",
  user="your_user",
  password="your_password",
  database="ecommerce"
)
cursor = conn.cursor(dictionary=True)
cursor.execute("SELECT * FROM inventory WHERE product_id = %s", (product_id,))
```

Service 3: Pricing Service (20% of service grade)

Type: Combination - receives parameters AND queries database^{[3][4]}

Responsibilities:

- Calculate final pricing based on product list from Order Service
- Query database for base prices, discount rules, and tax rates
- Apply promotional discounts and bulk pricing
- Return itemized pricing breakdown

Required Endpoints:

- POST /api/pricing/calculate - Calculate order total

Input from Order Service:

```
{
  "products": [
    {"product_id": 1, "quantity": 2},
    {"product_id": 3, "quantity": 1}
  ]
}
```

Database Tables:

```
CREATE TABLE pricing_rules (
  rule_id INT PRIMARY KEY AUTO_INCREMENT,
  product_id INT,
  min_quantity INT,
```

```

    discount_percentage DECIMAL(5,2)
);

CREATE TABLE tax_rates (
    region VARCHAR(50) PRIMARY KEY,
    tax_rate DECIMAL(5,2)
);

```

 **Hint:** This service should call the Inventory Service to get base prices, then apply discounts from the database. Use Python's requests library to make HTTP calls between services:

```

import requests
response = requests.get(f"http://localhost:5002/api/inventory/check/{product_id}")
product_data = response.json()

```

Service 4: Customer Service (15% of service grade)

Type: Combination - database queries AND external service calls

Responsibilities:

- Manage customer profile information from database
- Validate customer credentials
- Retrieve customer order history by calling Order Service
- Update customer loyalty points

Required Endpoints:

- GET /api/customers/{customer_id} - Get customer profile
- GET /api/customers/{customer_id}/orders - Get order history (calls Order Service)
- PUT /api/customers/{customer_id}/loyalty - Update loyalty points

Database Schema:

```

CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(20),
    loyalty_points INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

 **Hint:** When retrieving order history, first get the customer from your database, then make an HTTP GET request to the Order Service with a query parameter: `http://localhost:5001/api/orders?customer_id={customer_id}`

Service 5: Notification Service (15% of service grade)

Type: Combination - aggregates data from multiple services^{[10][3]}

Responsibilities:

- Receive notification requests with order ID
- Call Customer Service to get contact information
- Call Inventory Service to get delivery estimates

- Send email/SMS notification (simulate with console output)
- Log all notifications to database

Required Endpoints:

- POST /api/notifications/send - Send order notification

Service Communication Flow:

1. Receive order_id from Order Service
2. GET request → Customer Service (retrieve email/phone)
3. GET request → Inventory Service (check stock status)
4. Generate notification message
5. Log to database
6. Return success confirmation

Database Schema:

```
CREATE TABLE notification_log (
  notification_id INT PRIMARY KEY AUTO_INCREMENT,
  order_id INT NOT NULL,
  customer_id INT NOT NULL,
  notification_type VARCHAR(50),
  message TEXT,
  sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

 **Hint:** This service demonstrates service composition. You'll need to make multiple HTTP calls and combine the results. For email simulation, simply print to console:

```
print(f"EMAIL SENT TO: {customer_email}")
print(f"Subject: Order #{order_id} Confirmed")
print(f"Body: {notification_message}")
```

Java JSP Application Requirements

Frontend Pages (Required)

1. **index.jsp** - Product catalog display (read from Inventory Service)
2. **checkout.jsp** - Order form with customer and product selection
3. **confirmation.jsp** - Order success page with details

Servlet Implementation

Create servlets to handle communication with Flask services:^{[5][6]}

Example: OrderServlet.java

```
import java.net.http.*;
import java.net.URI;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/submitOrder")
public class OrderServlet extends HttpServlet {
  protected void doPost(HttpServletRequest request,
```

```

        HttpServletResponse response)
        throws ServletException, IOException {

    // Get form parameters
    String customerId = request.getParameter("customer_id");
    String productId = request.getParameter("product_id");
    String quantity = request.getParameter("quantity");

    // Build JSON payload
    String jsonPayload = String.format(
        "{\"customer_id\":%s,\"products\": [{\"product_id\":%s,\"quantity\":%s}]",
        customerId, productId, quantity
    );

    // Call Flask Order Service
    HttpClient client = HttpClient.newHttpClient();
    HttpRequest flaskRequest = HttpRequest.newBuilder()
        .uri(URI.create("http://localhost:5001/api/orders/create"))
        .header("Content-Type", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
        .build();

    try {
        HttpResponse<String> flaskResponse =
            client.send(flaskRequest, HttpResponse.BodyHandlers.ofString());

        // Forward to confirmation page
        request.setAttribute("orderResponse", flaskResponse.body());
        request.getRequestDispatcher("confirmation.jsp").forward(request, response);

    } catch (InterruptedException e) {
        response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
    }
}
}

```

 **Hint:** For NetBeans users, remember to add the Jakarta EE API library to your project dependencies. Use `HttpClient` (Java 11+) for REST calls instead of deprecated `URLConnection`.

Database Setup Instructions

Step 1: Create Database

```

CREATE DATABASE ecommerce_system;
USE ecommerce_system;

```

Step 2: Create All Required Tables

Execute all table creation SQL statements provided in each service specification above.

Step 3: Insert Sample Data

```

-- Sample products
INSERT INTO inventory (product_name, quantity_available, unit_price) VALUES
('Laptop', 50, 999.99),
('Mouse', 200, 29.99),
('Keyboard', 150, 79.99),
('Monitor', 75, 299.99),
('Headphones', 100, 149.99);

```

```
-- Sample customers
INSERT INTO customers (name, email, phone, loyalty_points) VALUES
('Ahmed Hassan', 'ahmed@example.com', '01012345678', 100),
('Sara Mohamed', 'sara@example.com', '01098765432', 250),
('Omar Ali', 'omar@example.com', '01055555555', 50);

-- Sample pricing rules
INSERT INTO pricing_rules (product_id, min_quantity, discount_percentage) VALUES
(1, 5, 10.00),
(2, 10, 15.00),
(3, 10, 12.00);
```

 **Hint:** Create a separate database user with limited privileges for security:

```
CREATE USER 'ecommerce_user'@'localhost' IDENTIFIED BY 'secure_password';
GRANT SELECT, INSERT, UPDATE ON ecommerce_system.* TO 'ecommerce_user'@'localhost';
FLUSH PRIVILEGES;
```

Implementation Guidelines

Phase 1: Setup (Week 1)

1. Install required software (Python, Java JDK, MySQL, Tomcat)
2. Create database and tables
3. Set up project structure for JSP and each Flask service
4. Test basic connectivity

 **Hint:** Create a virtual environment for each Python service to manage dependencies:

```
python3 -m venv service1_env
source service1_env/bin/activate # Linux/Mac
pip install flask mysql-connector-python requests
```